

Government Engineering College Kishanganj

Department of Computer Science and Engineering

Computer Networks LAB-04

Objective

To implement a TCP-based client-server application in C that performs query-based retrieval of student information.

Problem Statement

Design and implement a TCP client-server program in C with the following specifications:

1. The server maintains a text file containing student records. Each record consists of the following fields: **Registration Number, First Name (FName), Branch, and College Name.**
““
2. The client sends a **Registration Number** as a query to the server.
3. Upon receiving the query, the server performs the following operations:
 - Searches the database file for a matching Registration Number.
 - If a match is found, the server sends the corresponding **First Name, Branch, and College Name** back to the client.
 - If no match is found, the server sends the message:
“**Data not found**”.
4. The communication between client and server should be **continuous (persistent connection)**. The client can send multiple queries in a single session.
5. The session terminates only when the client sends the message “**bye**”, after which both client and server close the connection gracefully. ““

Sample Input/Output

Client Input: 1002 **Server Output:** Name: Amit, Branch: ECE, College: GEC_Kishanganj

Client Input: 9999 **Server Output:** Data not found

Client Input: bye **Output:** Connection terminated

Server Template

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

void find_student(char *reg_no, char *response) {
    FILE *fp = fopen("students.txt", "r");
    if (fp == NULL) {
        strcpy(response, "Error opening file");
        return;
    }

    char file_reg[50], name[50], branch[50], college[100];
    int found = 0;

    while (fscanf(fp, "%s%s%s%s", file_reg, name, branch, college) != EOF) {
        if (strcmp(file_reg, reg_no) == 0) {
            sprintf(response, "Name: %s, Branch: %s, College: %s", name, branch, college);
            found = 1;
            break;
        }
    }
}
```

```

    }

    if (!found) {
        strcpy(response, "Data_not_found");
    }

    fclose(fp);
}

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);

    char buffer[BUFFER_SIZE];
    char response[BUFFER_SIZE];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

----
----
----

    while (1) {
----
----
----

        find_student(buffer, response);

        send(new_socket, response, strlen(response), 0);
    }

    close(new_socket);
    close(server_fd);

    return 0;
}

```

Client Template

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;

    char reg_no[BUFFER_SIZE];
    char buffer[BUFFER_SIZE];

    -----
    -----
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr)
        ;

    -----
    -----
    -----

    while (1) {
        printf("\nEnter Registration Number (or type 'bye' to exit): ");
        fgets(reg_no, BUFFER_SIZE, stdin);
        reg_no[strcspn(reg_no, "\n")] = 0;

        send(sock, reg_no, strlen(reg_no), 0);

        // Exit condition
        if (strcmp(reg_no, "bye") == 0) {
```

```
        printf("Connection_closed.\n");
        break;
    }

    memset(buffer, 0, BUFFER_SIZE);
    read(sock, buffer, BUFFER_SIZE);

    printf("Server_Response:_%s\n", buffer);
}

close(sock);

return 0;
}
```